

Hiver SDE Intern — AI Support Agent for AppleSupport

1. Problem Framing

I chose **AppleSupport** as the target brand from the Customer Support on Twitter dataset. The goal was to build a small support agent that can take an incoming customer message and make three decisions: understand the customer's primary problem, draft a useful response based on how AppleSupport historically responded to similar issues, and decide whether the response is safe to automate or should be handed to a human.

For this project, I define a good system as one that is **correct about the customer's main issue, grounded in historical support behavior, and conservative when there is not enough evidence to respond safely.**

Intent classification

I created a 10-intent taxonomy, consisting of nine primary support intents plus an OTHER category, based on the types of problems present in AppleSupport interactions

- IOS_UPDATE
- DEVICE_PERFORMANCE
- BATTERY_POWER
- CONNECTIVITY
- APPS_APP_STORE
- APPLE_ID_ICLOUD
- MEDIA_SERVICES
- PAYMENTS_PURCHASES
- DATA_BACKUP_RESTORE
- OTHER

The classifier also receives previous context when available. This is important because Twitter support messages are often short follow-ups such as *"it is still happening"* or *"it's also draining my battery"*. Looking only at the latest message can therefore produce the wrong intent.

I intentionally classify the **primary support problem**, rather than simply matching keywords. For example, a customer mentioning battery drain in a follow-up to a black-screen/restarting problem should not automatically be classified as a battery issue if the surrounding context makes the underlying problem clear.

Historical grounding

For response generation, I extracted direct customer → AppleSupport interactions and built a historical retrieval layer using TF-IDF similarity. The retrieved examples are treated as **evidence of how AppleSupport historically handled similar cases**, not as proof that the historical customer was actually resolved.

The generated response is therefore instructed not to copy historical replies verbatim, invent policies, or claim that a problem is solved when the data does not establish that.

Auto-handle vs escalation

The final decision is intentionally separate from intent classification. A message can be correctly classified but still be unsuitable for automatic handling.

The current policy considers:

- intent confidence,
- availability of similar historical interactions,
- retrieval similarity,
- and higher-risk categories such as payment/purchase issues.

AUTO means there is enough confidence and historical evidence to send a bounded response. It **does not mean the customer's issue is guaranteed to be resolved**.

What I chose not to build

I deliberately kept the scope focused. I did not attempt to build a general Apple troubleshooting knowledge base, perform account or payment actions, guarantee resolution, or build a fully autonomous support system. I also did not attempt to process the full ~3M tweet dataset.

Instead, I focused on building and evaluating a complete prototype pipeline from **messy support data** → **intent** → **historical evidence** → **response** → **escalation**.

2. Results vs Baselines

I evaluated the system using a frozen set of **200 AppleSupport examples**. The examples were sampled to include representative, difficult, and edge cases. Initial labels were AI-assisted and then manually reviewed and corrected using explicit intent-boundary rules.

I compared the LLM classifier against two baselines: a trivial majority-class classifier and a simple TF-IDF + Logistic Regression classifier.

Model	Accuracy	Macro F1
Majority Class	17.0%	0.032
TF-IDF + Logistic Regression	75.0%	0.651
GPT-OSS-20B	61.5%*	0.543*

The LLM produced successful predictions for 195 of the 200 examples; five calls failed because of model/output issues.

The majority baseline predicts the most common intent, IOS_UPDATE, for every example. Its 17% accuracy provides a useful lower bound.

The TF-IDF + Logistic Regression model performed better than the LLM on this small holdout, achieving 75% accuracy and 0.651 Macro F1. However, the holdout contains only 40 examples, so I treat this as a directional comparison rather than a robust estimate.

The GPT-OSS-20B classifier achieved 61.5% accuracy and 0.543 Macro F1 on successful predictions. Its behavior was uneven across intents. It performed particularly well on APPLE_ID_ICLOUD, BATTERY_POWER, and CONNECTIVITY, while IOS_UPDATE had only 21% recall. This reflects an important property of the dataset: customers often mention an iOS update while actually describing a different underlying problem.

For the complete support-agent pipeline, **94 of 200 examples successfully completed response generation** before the Groq free-tier token quota was reached. These successful outputs were retained for evaluation rather than rerunning or changing the evaluation set.

The escalation policy achieved **70.2% accuracy**, but this number hides an important weakness: escalation recall was only **6.9% (2/29)**. The policy therefore needs significant improvement before automatic handling could be trusted for broader use.

3. Failure Analysis

3.1 Escalation policy is too conservative

The biggest weakness is the escalation decision.

The policy achieved 70.2% overall accuracy but identified only **2 of 29 cases** that the reviewer marked for escalation. It therefore incorrectly selected AUTO for **27 escalation cases**.

The likely reason is that the current policy relies heavily on intent confidence, retrieval similarity, and a small number of explicit risk rules. These signals do not adequately capture repeated complaints, unresolved troubleshooting, account-specific problems, or situations that require human intervention.

Hypothesis: confidence and retrieval quality are useful signals, but they are not sufficient to determine whether an issue is safe to automate.

Next step: add explicit signals for unresolved previous attempts, repeated complaints, account/access problems, hardware/service issues, and other high-risk situations. I would optimize primarily for **escalation recall**, rather than raw accuracy.

3.2 Intent boundary confusion

The classifier had strong recall for DEVICE_PERFORMANCE at **88%**, but only **21% recall for IOS_UPDATE**.

A representative example is a customer whose latest message mentions battery drain, while the previous context describes repeated black screens, loading wheels, and passcode prompts after an update.

A keyword-based system may choose BATTERY_POWER, while the conversation-level interpretation is DEVICE_PERFORMANCE.

Hypothesis: customers frequently mention multiple symptoms and product context. The latest keyword is not always the customer's primary support problem.

Next step: add more hard examples around intent boundaries and continue using previous context explicitly during classification.

3.3 Retrieval similarity does not guarantee grounded generation

GS-100 is a useful example. The retriever produced a similarity score of **1.0**, but the generated response still introduced a specific troubleshooting recommendation that was not sufficiently supported by the retrieved historical evidence.

This showed that a high retrieval score alone does not guarantee a grounded answer.

Hypothesis: retrieval finds relevant examples, but the LLM can still extrapolate beyond those examples while generating the final response.

Next step: make recommendations traceable to retrieved evidence and add an explicit **insufficient-evidence** path instead of allowing the model to fill gaps with plausible-sounding advice.

3.4 The LLM judge is optimistic about groundedness

I compared the LLM-as-judge against reviewer scores on a 25-example calibration subset. Overall, the judge's score was within one point of the reviewer score for 84% of examples. Agreement was 84% for helpfulness, 80% for relevance, and 76% for groundedness. The largest disagreement was groundedness: the reviewer average was 3.48/5, while the LLM judge average was 4.80/5. This suggests that the judge is more optimistic about whether responses are sufficiently grounded.

GS-100 again exposed this limitation: the reviewer considered the response insufficiently grounded, while the LLM judge rated it highly.

Hypothesis: the judge is better at recognizing whether a response sounds relevant and helpful than checking whether each recommendation is actually supported by the retrieved evidence.

Next step: use a stricter evidence-entailment rubric and include adversarial examples where a response sounds reasonable but contains unsupported claims.

3.5 LLM provider limits reduced response evaluation coverage

Response generation successfully completed for **94/200 examples**. The remaining 106 were primarily affected by the Groq free-tier daily token quota, along with some earlier output-length failures.

This is not a model-quality failure, but it is an evaluation limitation because response-quality results are based on a partial set.

Hypothesis: sequential LLM calls containing retrieved context consume the provider token budget quickly.

Next step: cache model calls, reduce prompt/output size, and make the evaluation harness provider-independent.

4. What is misleading about my headline number?

The most tempting headline number is the **61.5% intent accuracy**, but it should not be interpreted as production-level model accuracy.

First, it represents **195 successful predictions rather than all 200 examples** because five model calls failed. Second, the OTHER intent has zero examples in the golden set, so performance for that class cannot be evaluated. Third, the golden labels were **AI-assisted and manually reviewed**, rather than independently double-annotated human ground truth. Finally, the TF-IDF baseline's 75% accuracy comes from only a 40-example holdout, so that number also has high variance.

The escalation result has a different issue. Its 70.2% accuracy looks reasonable at first glance, but escalation recall is only 6.9%. Because AUTO is the majority class, overall accuracy hides the fact that the current policy misses most cases that should be escalated.

Therefore, I view these results as **directional prototype measurements rather than production performance estimates**. The more useful conclusion is not that the agent is ready for autonomous support, but that the evaluation identified exactly where the system still needs work.

5. What I Would Do With One More Week

With another week, I would focus less on increasing the headline intent accuracy and more on making the complete system safer and easier to evaluate.

1. Improve escalation-I would redesign the escalation policy around **high recall for cases that should reach a human**. I would add features for unresolved previous attempts, repeated complaints, account/access issues, transactional problems, hardware/service problems, and customer frustration.

2. Improve retrieval-I would replace the TF-IDF-only retriever with semantic embeddings and reranking. This should help retrieve historical examples where the wording differs but the underlying support problem is similar.

3. Make generation evidence-aware-Every recommendation in a generated response should be traceable to retrieved evidence. If the historical data does not provide enough support, the system should ask for more information or escalate instead of inventing troubleshooting steps.

4. Strengthen evaluation-I would expand the golden set and introduce independent human annotation so that inter-annotator agreement can be measured. I would also use a fixed split or cross-validation for the simple baseline and report intent macro F1, per-intent recall, escalation recall, and response groundedness/helpfulness.

5. Improve robustness-Finally, I would test the pipeline on long conversations, ambiguous follow-ups, multiple symptoms, missing context, noisy text, repeated complaints, and cases where historical evidence is weak. I would also add caching and rate-limit handling to make repeated evaluations reliable.

6. Decision Log

1. **Selected AppleSupport** because it provided a sufficiently large set of direct customer-support interactions and enough variety for intent discovery.
2. **Used direct customer → support interactions** as the primary unit instead of treating every connected Twitter graph component as a single conversation.
3. **Validated parent-child relationships** before reconstructing interactions because the dataset contains missing parent references.
4. **Did not treat connected components as true conversations** after observing large Twitter threads containing unrelated customer messages.
5. **Created a small nine-intent taxonomy** rather than trying to model every possible support topic.
6. **Made intents problem-oriented** so that the system predicts the customer's primary issue instead of simply matching keywords.
7. **Included previous context in classification** because many Twitter messages are short follow-ups.
8. **Separated classification from escalation** because a correctly classified issue may still require human handling.
9. **Used historical responses as evidence rather than proof of resolution** because a support reply does not establish that the customer's issue was ultimately solved.
10. **Started with TF-IDF retrieval** because it is simple, interpretable, fast, and provides a transparent historical-retrieval baseline.
11. **Added explicit escalation for payment/purchase issues** because transactional cases are higher risk for autonomous handling.
12. **Used an LLM judge as an evaluation tool rather than ground truth** and compared it against reviewer scores to identify weaknesses in the judge itself.